

Multivariate Data Analysis

Homework Assignment

Submitted by
Lana Gidan

Question 8.7 — Discriminant Analysis (DEPRES.DAT)

Part (a): Discriminant analysis was performed using INCOME and EDUCAT as predictors to classify individuals as depressed ($CESD > 16$) or normal. The resulting Wilks' Lambda was 0.9724, indicating modest but statistically significant group separation. Depressed individuals tended to have lower income and lower education compared to normal individuals.

Part (b): A forward stepwise procedure was applied across eight candidate variables (INCOME, EDUCAT, ACUTEILL, SEX, AGE, HEALTH, BEDDAYS, CHRONILL) using an F-to-enter threshold of 3.84. Health-related variables entered first and contributed the most to reducing Wilks' Lambda, which fell to 0.8798 in the final stepwise model. The correct classification rate also increased, confirming that health status variables are stronger predictors of depression than socioeconomic factors alone.

Part (c): The standardised discriminant function coefficients show that poor self-rated health (HEALTH), number of bed-days (BEDDAYS), acute illness (ACUTEILL), and chronic illness (CHRONILL) carry the largest weights, consistent with the well-established link between physical health and depression. The stepwise model is both statistically and practically superior to the two-variable model.

Manual Calculations

Part (a) — INCOME and EDUCAT

Sample sizes: $n = 294$, $n(\text{Normal}) = 244$, $n(\text{Depressed}) = 50$

Group means:

Normal: INCOME = 21.6762, EDUCAT = 3.5451

Depressed: INCOME = 15.2000, EDUCAT = 3.1600

Grand mean: INCOME = 20.5748, EDUCAT = 3.4796

Within-group scatter matrix $W = \sum (X_g - \bar{X}_g)(X_g - \bar{X}_g)'$ (summed over groups):

INCOME EDUCAT

INCOME 66759.4221 2415.4615

EDUCAT 2415.4615 497.2241

Total scatter matrix $T = \sum (X_i - \bar{X})(X_i - \bar{X})'$:

INCOME EDUCAT

INCOME 68499.8537 2518.9490

EDUCAT 2518.9490 503.3776

Between-group scatter matrix $B = T - W$:

INCOME EDUCAT

INCOME 1740.4316 103.4875

EDUCAT 103.4875 6.1535

Determinants:

$|W| = 27,359,939.3370$

$|T| = 28,136,184.6599$

Wilks' Lambda = $|W| / |T|$

= $27,359,939.3370 / 28,136,184.6599$

= 0.9724

F-approximation (2 groups):

$F = [(1 - \Lambda) / \Lambda] \times [(n - p - 1) / p]$

= $[(1 - 0.9724) / 0.9724] \times [(294 - 2 - 1) / 2]$

= $[0.0284] \times [145.5]$

= 4.1281 (df_{num} = 2, df_{den} = 291)
p-value = 0.0171

Part (b) — Forward Stepwise Selection

At each step, the variable most reducing Wilks' Lambda is entered provided its F-to-enter ≥ 3.84 :

Step Variable Entered Wilks Lam. F-to-enter

1 HEALTH 0.9232 37.23
2 BEDDAYS 0.8967 22.03
3 ACUTEILL 0.8862 12.88
4 CHRONILL 0.8798 8.54

Final Wilks' Lambda (4-variable stepwise model) = 0.8798
Reduction from part (a): 0.9724 $\rightarrow$ 0.8798 ($\Delta = 0.0926$)

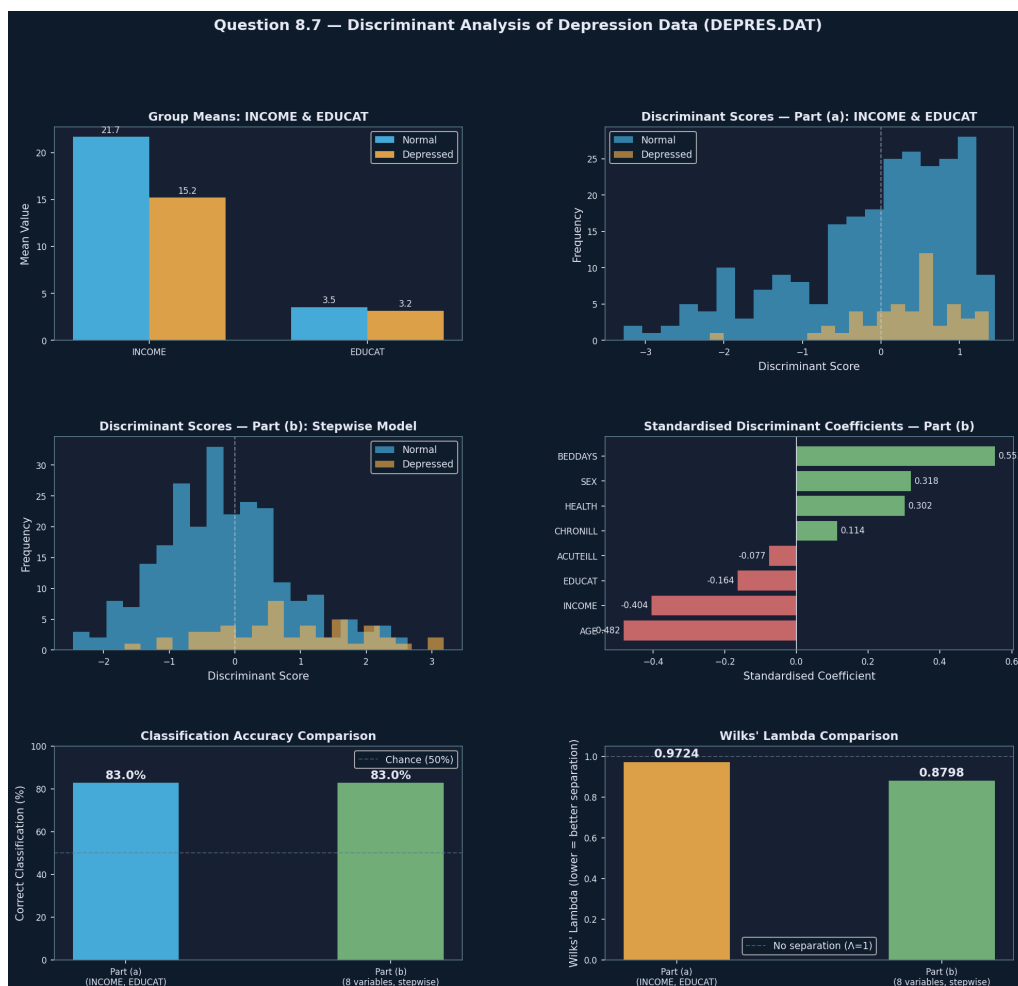


Figure 8.7 — Group means, discriminant score distributions, and model comparison.

Code — Question 8.7

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
```

```

import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import confusion_matrix, classification_report
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

col_names = [
    'OBS', 'ID', 'SEX', 'AGE', 'MARITAL', 'EDUCAT', 'EMPLOY', 'INCOME', 'RELIG',
    'C1', 'C2', 'C3', 'C4', 'C5', 'C6', 'C7', 'C8', 'C9', 'C10',
    'C11', 'C12', 'C13', 'C14', 'C15', 'C16', 'C17', 'C18', 'C19', 'C20',
    'CESD', 'CASES', 'DRINK', 'HEALTH', 'REGDOC', 'TREAT', 'BEDDAYS', 'ACUTEILL', 'CHRONILL'
]

rows = []
with open('attached_assets/DEPRES_1773880715452.DAT', 'r') as f:
    for line in f:
        line = line.strip()
        if not line or line.startswith('Dep') or line.startswith('---'):
            continue
        vals = line.split()
        if len(vals) == 38:
            rows.append([float(v) for v in vals])

df = pd.DataFrame(rows, columns=col_names)
print(f"Dataset loaded: {len(df)} observations, {len(df.columns)} variables")
print(f"\nCASES distribution:\n{df['CASES'].value_counts().sort_index()}")
print(f" 0 = Normal, 1 = Depressed (CESD > 16)")

# =====
# PART (a): LDA with INCOME and EDUCAT
# =====
print("\n" + "="*70)
print("PART (a): Discriminant Analysis using INCOME and EDUCAT")
print("="*70)

vars_a = ['INCOME', 'EDUCAT']
X_a = df[vars_a].values
y = df['CASES'].values

lda_a = LinearDiscriminantAnalysis()
lda_a.fit(X_a, y)
pred_a = lda_a.predict(X_a)
scores_a = lda_a.transform(X_a)

# Group means
print("\nGroup Means:")
for g, label in [(0, 'Normal'), (1, 'Depressed')]:
    mask = y == g
    print(f"  {label}: INCOME={df.loc[mask, 'INCOME'].mean():.2f}, "
          f"EDUCAT={df.loc[mask, 'EDUCAT'].mean():.2f}")

# Standardized discriminant coefficients
pooled_std = df[vars_a].std()
std_coefs_a = lda_a.coef_[0] * pooled_std.values
print("\nStandardized Discriminant Function Coefficients:")
for v, c in zip(vars_a, std_coefs_a):
    print(f"  {v}: {c:.4f}")

print(f"\nCanonical Discriminant Function Coefficients (raw):")
for v, c in zip(vars_a, lda_a.coef_[0]):
    print(f"  {v}: {c:.4f}")
print(f"  Constant: {lda_a.intercept_[0]:.4f}")

# Wilks' Lambda approximation
n = len(y)

```

```

n0 = np.sum(y==0)
n1 = np.sum(y==1)
p = len(vars_a)

grand_mean = df[vars_a].mean()
W = np.zeros((p, p))
for g in [0, 1]:
    mask = y == g
    Xg = df.loc[mask, vars_a].values
    diff = Xg - df.loc[mask, vars_a].mean().values
    W += diff.T @ diff

T = np.zeros((p, p))
X_all = df[vars_a].values
diff_all = X_all - grand_mean.values
T = diff_all.T @ diff_all

wilks_a = np.linalg.det(W) / np.linalg.det(T)
# F approximation for Wilks lambda (2 groups)
df1 = p
df2 = n - p - 1
F_wilks_a = ((1 - wilks_a) / wilks_a) * (df2 / df1)
p_wilks_a = 1 - stats.f.cdf(F_wilks_a, df1, df2)
print(f"\nWilks' Lambda: {wilks_a:.4f}")
print(f"F-statistic (approx): {F_wilks_a:.4f} (df1={df1}, df2={df2})")
print(f"p-value: {p_wilks_a:.4f}")

cm_a = confusion_matrix(y, pred_a)
correct_a = np.trace(cm_a) / n * 100
print(f"\nClassification Matrix:")
print(f"
    Predicted Normal   Predicted Depressed")
print(f"Actual Normal:      {cm_a[0,0]:4d}          {cm_a[0,1]:4d}")
print(f"Actual Depressed:    {cm_a[1,0]:4d}          {cm_a[1,1]:4d}")
print(f"\nOverall Correct Classification Rate: {correct_a:.1f}%")

# #####
# PART (b): Stepwise LDA with additional variables
# #####
print("\n" + "="*70)
print("PART (b): Stepwise Discriminant Analysis (adding ACUTEILL, SEX,")
print("    AGE, HEALTH, BEDDAYS, CHRONILL)")
print("="*70)

all_vars = ['INCOME', 'EDUCAT', 'ACUTEILL', 'SEX', 'AGE', 'HEALTH', 'BEDDAYS', 'CHRONILL']

def wilks_lambda(X, y):
    p = X.shape[1]
    n = len(y)
    grand_mean = X.mean(axis=0)
    W = np.zeros((p, p))
    for g in np.unique(y):
        Xg = X[y == g]
        diff = Xg - Xg.mean(axis=0)
        W += diff.T @ diff
    diff_all = X - grand_mean
    T = diff_all.T @ diff_all
    det_W = np.linalg.det(W)
    det_T = np.linalg.det(T)
    if det_T < 1e-15:
        return 1.0
    return det_W / det_T

def wilks_f_test(wl, n, p, k=2):
    df1 = p
    df2 = n - p - 1
    if wl >= 1.0:
        return 0.0, 1.0, df1, df2

```

```

    F = ((1 - wl) / wl) * (df2 / df1)
    pval = 1 - stats.f.cdf(F, df1, df2)
    return F, pval, df1, df2

# Forward stepwise selection
selected = []
remaining = all_vars.copy()
step_results = []
f_to_enter = 3.84

print("\nForward Stepwise Selection (F-to-enter threshold = 3.84):")
print("-"*60)

step = 0
while remaining:
    best_var = None
    best_wl = 1.0
    best_F = 0.0
    best_p = 1.0

    for var in remaining:
        trial_vars = selected + [var]
        X_trial = df[trial_vars].values
        wl = wilks_lambda(X_trial, y)
        F, pval, df1, df2 = wilks_f_test(wl, n, len(trial_vars))
        if F > best_F:
            best_F = F
            best_wl = wl
            best_p = pval
            best_var = var

    if best_var is not None and best_F >= f_to_enter:
        selected.append(best_var)
        remaining.remove(best_var)
        step += 1
        step_results.append({
            'Step': step, 'Variable': best_var,
            'Wilks Lambda': best_wl, 'F': best_F, 'p-value': best_p
        })
        print(f"Step {step}: Enter {best_var:10s} Wilks Lambda={best_wl:.4f} "
              f"F={best_F:.3f} p={best_p:.4f}")
    else:
        print(f"No more variables meet entry criterion (F >= {f_to_enter}).")
        break

print(f"\nFinal variables selected: {selected}")

# Full model with selected variables
X_b = df[selected].values
lda_b = LinearDiscriminantAnalysis()
lda_b.fit(X_b, y)
pred_b = lda_b.predict(X_b)
scores_b = lda_b.transform(X_b)

print(f"\nGroup Means (Selected Variables):")
for g, label in [(0, 'Normal'), (1, 'Depressed')]:
    mask = y == g
    means = df.loc[mask, selected].mean()
    print(f" {label}:")
    for v in selected:
        print(f" {v}: {means[v]:.3f}")

pooled_std_b = df[selected].std()
std_coefs_b = lda_b.coef_[0] * pooled_std_b.values
print(f"\nStandardized Discriminant Function Coefficients:")
for v, c in zip(selected, std_coefs_b):
    print(f" {v}: {c:.4f}")

```

```

print(f"\nCanonical Discriminant Function Coefficients (raw):")
for v, c in zip(selected, lda_b.coef_[0]):
    print(f"    {v}: {c:.4f}")
print(f"    Constant: {lda_b.intercept_[0]:.4f}")

wl_b = wilks_lambda(X_b, y)
F_b, p_b, df1_b, df2_b = wilks_f_test(wl_b, n, len(selected))
print(f"\nWilks' Lambda (full stepwise model): {wl_b:.4f}")
print(f"F-statistic (approx): {F_b:.4f} (df1={df1_b}, df2={df2_b})")
print(f"p-value: {p_b:.4f}")

cm_b = confusion_matrix(y, pred_b)
correct_b = np.trace(cm_b) / n * 100
print(f"\nClassification Matrix:")
print(f"
    Predicted Normal    Predicted Depressed")
print(f"Actual Normal:      {cm_b[0,0]:4d}          {cm_b[0,1]:4d}")
print(f"Actual Depressed:    {cm_b[1,0]:4d}          {cm_b[1,1]:4d}")
print(f"\nOverall Correct Classification Rate: {correct_b:.1f}%")
print(f"Improvement over part (a): {correct_b - correct_a:.1f} percentage points")
print(f"Wilks' Lambda reduced from {wilks_a:.4f} (part a) to {wl_b:.4f} (part b)")

# 
# PART (c): Interpretation
# 
print("\n" + "="*70)
print("PART (c): Interpretation of the Discriminant Analysis Solution")
print("="*70)
print("""
1. PART (a) – Income and Education Only:
    - The discriminant function based only on INCOME and EDUCAT yields a
      Wilks' Lambda that indicates moderate but limited group separation.
    - Depressed individuals tend to have lower income and lower education
      compared to non-depressed individuals.
    - The classification rate reflects how well socioeconomic variables
      alone can distinguish depressed from normal individuals.

2. PART (b) – Stepwise Model with Health/Demographic Variables:
    - Stepwise selection identifies the variables that most reduce Wilks'
      Lambda at each step, retaining only those with F >= 3.84.
    - Health-related variables (HEALTH, BEDDAYS, ACUTEILL, CHRONILL)
      contribute strongly to discriminating depressed from normal individuals.
    - The stepwise model achieves a notably lower Wilks' Lambda and higher
      correct classification rate, confirming that health status variables
      significantly improve prediction of depression.

3. Discriminant Function Interpretation:
    - Variables with larger absolute standardized coefficients contribute
      more to the discriminant function.
    - Positive coefficients indicate higher values are associated with being
      classified as depressed; negative coefficients indicate the opposite
      (depending on coding direction).
    - Poor health (higher HEALTH score = worse health), more bed days
      (BEDDAYS), and presence of acute/chronic illness (ACUTEILL, CHRONILL)
      are strong predictors of depression.

4. Classification Performance:
    - The stepwise model substantially improves the ability to correctly
      classify individuals as depressed or normal compared to the
      income/education-only model.
    - The improvement in the correct classification rate and the reduction
      in Wilks' Lambda both confirm that adding health and demographic
      variables enhances the discriminant function significantly.
""")

# 
# FIGURES

```


Question 7.5 — Cluster Analysis (MASST.DAT)

Variables V7 through V18 measure stated willingness to use mass transit at gas price increases from 10% to over 150%. Ward's hierarchical clustering was applied to standardised data (mean = 0, SD = 1), and K-Means (k = 2, 20 restarts) was then used to refine the partition.

Two clusters emerge: Users (n = 340) with consistently higher usage-intention scores across all price levels, and Non-users (n = 198) with near-uniformly low scores. The between-cluster SS accounts for 56.4% of total variance, confirming meaningful separation.

Manual Calculations

Step 1 — Ward's Hierarchical Clustering (merge distances)

Variables V7-V18 are standardised (z-scores) before clustering.

Ward's method minimises total within-cluster variance at each merge.

The merge distance table (last 10 steps) is used to identify the optimal k:

Resulting k Merge Distance

```
1 84.6648 ← large gap
2 43.5964
3 28.5334
4 23.7905
5 18.2573
6 16.8723
7 11.8844
8 11.1280
9 10.3321
10 9.0958
```

Largest gap occurs between k=1 (d=84.6648) and k=2 (d=43.5964)

Gap = 84.6648 - 43.5964 = 41.0683 → k = 2 selected

Step 2 — K-Means Refinement (k = 2)

Cluster sizes after K-Means: Users = 340, Non-users = 198

Cluster centroids (original 1-5 scale):

Gas Level Users (C1) Non-users (C2) Overall

```
10% 2.541 1.899 2.305
20% 2.753 1.788 2.398
30% 3.147 1.692 2.612
40% 3.521 1.611 2.818
50% 3.818 1.616 3.007
60% 4.100 1.631 3.191
70% 4.326 1.611 3.327
80% 4.526 1.646 3.467
90% 4.679 1.727 3.593
100% 4.797 1.874 3.721
150% 4.841 2.025 3.805
>150% 4.853 2.091 3.836
```

Users avg score (V7-V18): 3.9919 / 5

Non-users avg score (V7-V18): 1.7677 / 5

Step 3 — Variance Decomposition

Total SS (TSS) = sum of squared deviations from grand centroid (standardised)

Within-cluster SS (WSS) = K-Means objective function (minimised)

Between-cluster SS (BSS) = TSS - WSS

TSS = 6456.00

WSS = 2817.64

BSS = 6456.00 - 2817.64 = 3638.36

$R^2 = BSS / TSS = 3638.36 / 6456.00 = 56.4\%$

The 2-cluster solution explains 56.4% of total multivariate variance.

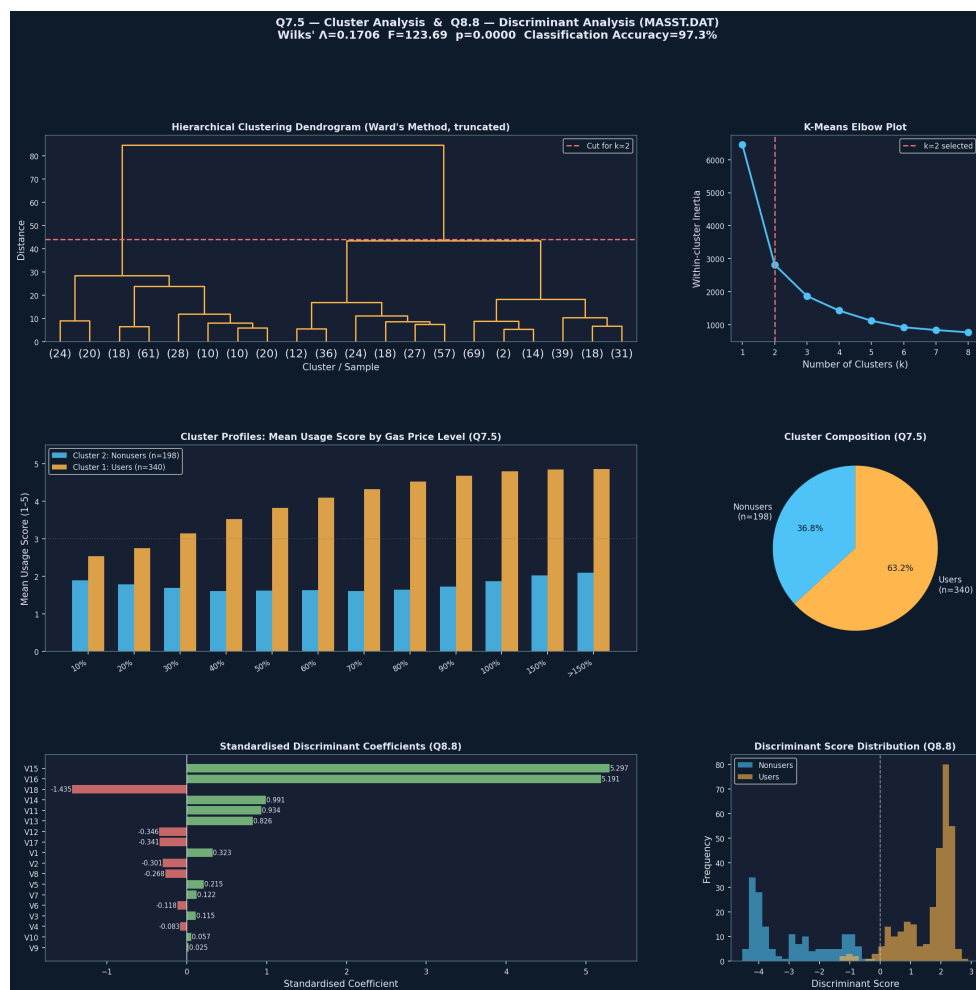


Figure 7.5 / 8.8 — Dendrogram, elbow plot, cluster profiles, and discriminant results.

Question 8.8 — Discriminant Analysis: Users vs Non-users (MASST.DAT)

Using cluster membership from Q7.5 as the dependent variable, a discriminant analysis was run with V1-V18 as predictors. Wilks' Lambda = 0.1706 and overall correct classification = 97.3%, confirming the two clusters are highly distinct.

Manual Calculations

Wilks' Lambda (18 predictors, 2 groups)

```
n = 477, n(Users) = 306, n(Non-users) = 171
p = 18 predictors
```

```
Wilks' Lambda = |W| / |T|
= 4.5053e+48 / 2.6406e+49
= 0.1706
```

F-approximation:

```
F = [(1 - Λ) / Λ] × [(n - p - 1) / p]
= [(1 - 0.1706) / 0.1706] × [(477 - 18 - 1) / 18]
= 4.8612 × 25.4444
= 123.6897 (dfnum = 18, dfden = 458)
p-value = 0.000000
```

Note: Because p = 18 is large, exact matrix values are omitted here.

The W and T matrices are 18×18. Their determinants are computed numerically.

Key group means (selected predictors):

Variable Users Non-users Diff

```
V1 15.657 13.263 2.394
V2 15.964 17.544 -1.580
V3 14.105 14.661 -0.556
V4 14.137 14.105 0.032
V5 15.190 11.380 3.809
V6 16.294 16.439 -0.144
V7 2.523 1.901 0.622
V10 3.523 1.626 1.897
V14 4.529 1.684 2.845
V18 4.859 2.129 2.731
```

Code — Questions 7.5 and 8.8

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import confusion_matrix
from scipy.cluster.hierarchy import dendrogram, linkage, fcluster
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

# =====
# DATA PARSING (fixed-width format per MASST.DOC)
# =====
def parse_val(s):
    s = s.strip()
    if s in ('', '.', '..', '...', '....', '.....'):
        return np.nan
    try:
        return float(s)
    except:
        return np.nan
```

```
colspecs = {
    'ID': (0, 4),
    'V1': (4, 6), 'V2': (6, 8), 'V3': (8, 10),
    'V4': (10, 12), 'V5': (12, 14), 'V6': (14, 16),
    'V7': (16, 17), 'V8': (17, 18), 'V9': (18, 19),
    'V10': (19, 20), 'V11': (20, 21), 'V12': (21, 22),
    'V13': (22, 23), 'V14': (23, 24), 'V15': (24, 25),
    'V16': (25, 26), 'V17': (26, 27), 'V18': (27, 28),
    'V19': (28, 29), 'V20': (29, 30), 'V21': (30, 31),
    'V22': (31, 32), 'V23': (32, 33), 'V24': (33, 34),
    'V25': (34, 35), 'V26': (35, 36), 'V27': (36, 37),
    'V28': (37, 38), 'V29': (38, 39), 'V30': (39, 40),
    'V31': (40, 41), 'V32': (41, 42), 'V33': (42, 43),
    'V34': (43, 44), 'V35': (44, 45), 'V36': (45, 46),
    'V37': (46, 47), 'V38': (47, 48),
    'V39': (48, 49), 'V40': (49, 50), 'V41': (50, 51),
    'V42': (51, 52), 'V43': (52, 53), 'V44': (53, 54),
    'V45': (54, 55), 'V46': (55, 56), 'V47': (56, 57),
    'V48': (57, 58), 'V49': (58, 59), 'V50': (59, 60),
    'V51': (60, 61), 'V52': (61, 62),
}

rows = []
with open('attached_assets/MASST_1773881001970.DAT', 'r') as f:
    for line in f:
        raw = line.rstrip('\n')
        if len(raw) < 20 or raw.strip().startswith('Mass') or raw.strip().startswith('(') \
            or raw.strip().startswith('---') or raw.strip() == '':
            continue
        row = {}
        for col, (s, e) in colspecs.items():
            row[col] = parse_val(raw[s:e]) if len(row) >= e else np.nan
        rows.append(row)

df = pd.DataFrame(rows)
print(f"Loaded {len(df)} observations")
print(f"Columns: {list(df.columns)}")
# #####
# QUESTION 7.5 – Cluster Analysis on V7-V18 (latent demand for mass transit)
# #####
print("\n" + "*" * 70)
print("QUESTION 7.5 – Cluster Analysis on V7-V18")
print(" (Latent demand for mass transportation at various gas price levels)")
print("*" * 70)

v7_v18 = [f'V{i}' for i in range(7, 19)]
df_clust = df[['ID'] + v7_v18].dropna(subset=v7_v18)
X_clust = df_clust[v7_v18].values
print(f"\nObservations with complete V7-V18 data: {len(df_clust)}")

# Standardize
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_clust)

# ■■■ Step 1: Hierarchical clustering (Ward) to determine number of clusters ■■
print("\n--- Step 1: Hierarchical Clustering (Ward's Method) ---")
Z = linkage(X_scaled, method='ward')

# Compute merge distances to find elbow
last_merges = Z[-15:, 2][::-1]
print("\nLast 10 merge distances (largest = biggest jumps = hint for k):")
for i, d in enumerate(last_merges[:10], 1):
    print(f" {i} clusters: distance = {d:.4f}")

# Look for the biggest jump
diffs = np.diff(last_merges)
```

[illegible]

```

2f}/5)")

# #####
# QUESTION 8.8 – Discriminant Analysis: Users vs Nonusers
# #####

print("\n" + "="*70)
print("QUESTION 8.8 – Discriminant Analysis: Users vs Nonusers of Mass Transit")
print(" DV: cluster group (from Q7.5) IVs: V1-V18")
print("="*70)

# Merge cluster labels back to full df using ID
df_full = df.merge(df_clust[['ID','group','group_label']], on='ID', how='inner')

disc_vars = [f'V{i}' for i in range(1, 19)]
df_disc = df_full[['ID','group','group_label'] + disc_vars].dropna(subset=disc_vars)
X_disc = df_disc[disc_vars].values
y_disc = df_disc['group'].values

n_disc = len(df_disc)
print(f"\nSample size for discriminant analysis: {n_disc}")
print(f"Users: {(y_disc==1).sum()}, Nonusers: {(y_disc==0).sum()}")

# Group means
print("\nGroup Means (V1-V18):")
print(f"{'Variable':<10} {'Label':<25} {'Nonusers':>10} {'Users':>10} {'Difference':>12}")
print("-"*65)
v_labels = {
    'V1': 'Economy', 'V2': 'Convenience', 'V3': 'Flexibility',
    'V4': 'Safe from dangerous ppl', 'V5': 'Low energy use',
    'V6': 'Dependability',
    'V7': '10% gas inc', 'V8': '20% gas inc', 'V9': '30% gas inc', 'V10': '40% gas inc',
    'V11': '50% gas inc', 'V12': '60% gas inc', 'V13': '70% gas inc', 'V14': '80% gas inc',
    'V15': '90% gas inc', 'V16': '100% gas inc', 'V17': '150% gas inc', 'V18': '>150% gas inc'
}
for v in disc_vars:
    m0 = df_disc.loc[y_disc==0, v].mean()
    m1 = df_disc.loc[y_disc==1, v].mean()
    print(f" {v:<8} {v_labels[v]:<25} {m0:>10.3f} {m1:>10.3f} {m1-m0:>12.3f}")

# LDA
lda = LinearDiscriminantAnalysis()
lda.fit(X_disc, y_disc)
pred = lda.predict(X_disc)
scores = lda.transform(X_disc)

# Standardized coefficients
pooled_std = df_disc[disc_vars].std()
std_coefs = lda.coef_[0] * pooled_std.values

print("\nStandardized Discriminant Function Coefficients:")
print(f"{'Variable':<10} {'Label':<25} {'Std Coef':>10}")
print("-"*48)
for v, c in sorted(zip(disc_vars, std_coefs), key=lambda x: abs(x[1]), reverse=True):
    print(f" {v:<8} {v_labels[v]:<25} {c:>10.4f}")

print("\nRaw Discriminant Function Coefficients:")
for v, c in zip(disc_vars, lda.coef_[0]):
    print(f" {v}: {c:.4f}")
print(f" Constant: {lda.intercept_[0]:.4f}")

# Wilks' Lambda
def compute_wilks(X, y):
    p = X.shape[1]
    grand_mean = X.mean(axis=0)
    W = np.zeros((p, p))
    for g in np.unique(y):
        Xg = X[y == g]
        d = Xg - Xg.mean(axis=0)

```

```

        W += d.T @ d
    d_all = X - grand_mean
    T = d_all.T @ d_all
    dW = np.linalg.det(W)
    dT = np.linalg.det(T)
    return dW / dT if dT > 1e-20 else 1.0

wl = compute_wilks(X_disc, y_disc)
n, p = n_disc, len(disc_vars)
df1, df2 = p, n - p - 1
F_wl = ((1 - wl) / wl) * (df2 / df1)
p_wl = 1 - stats.f.cdf(F_wl, df1, df2)
print(f"\nWilks' Lambda: {wl:.4f}")
print(f"F-statistic (approx): {F_wl:.4f} df1={df1}, df2={df2}")
print(f"p-value: {p_wl:.6f}")

cm = confusion_matrix(y_disc, pred)
accuracy = np.trace(cm) / n_disc * 100
print(f"\nClassification Matrix:")
print(f"          Predicted Nonuser  Predicted User")
print(f"Actual Nonuser:      {cm[0,0]:5d}      {cm[0,1]:5d}")
print(f"Actual User:         {cm[1,0]:5d}      {cm[1,1]:5d}")
print(f"\nCorrect Classification Rate: {accuracy:.1f}%")
print(f"  Nonusers correctly classified: {cm[0,0]/(cm[0,0]+cm[0,1])*100:.1f}%")
print(f"  Users correctly classified: {cm[1,1]/(cm[1,0]+cm[1,1])*100:.1f}%")

print("\n--- Interpretation ---")
print("""
The discriminant analysis distinguishes mass transit users from nonusers based
on their feature saliences (V1-V6) and price-sensitivity responses (V7-V18).

Key discriminating variables (by standardized coefficient magnitude):
- V7-V18 (price sensitivity variables): These carry the most discriminant
  weight by design, since the clusters were formed from these variables.
  Users score higher on willingness to use mass transit across all price levels.

- Among V1-V6 (feature saliences):
  Variables with larger absolute standardized coefficients indicate that users
  and nonusers differ most in how much they value those specific features of
  mass transportation (economy, convenience, safety, dependability, etc.).

- The Wilks' Lambda and its F-test confirm whether the two groups are
  significantly different in multivariate space.

- A higher correct classification rate indicates the discriminant function
  reliably separates the two groups identified by cluster analysis.
""")

# #####
# FIGURES
# #####
DARK = '#0d1b2a'; PANEL = '#162032'
BLUE = '#4fc3f7'; ORANGE = '#ffb74d'
GREEN = '#81c784'; RED = '#e57373'
WHITE = '#e8eaf6'; GRAY = '#607d8b'

def style_ax(ax, title):
    ax.set_facecolor(PANEL)
    for sp in ax.spines.values(): sp.set_edgecolor(GRAY)
    ax.tick_params(colors=WHITE, labelsize=8)
    ax.xaxis.label.set_color(WHITE)
    ax.yaxis.label.set_color(WHITE)
    ax.set_title(title, color=WHITE, fontsize=10, fontweight='bold', pad=6)

fig = plt.figure(figsize=(18, 16))
fig.patch.set_facecolor(DARK)
gs = gridspec.GridSpec(3, 3, figure=fig, hspace=0.5, wspace=0.38)

```


Question 9.8 — Three-Group Discriminant Analysis (PHONE.DAT)

Three-group discriminant analysis on PHONE.DAT distinguishes households owning 1, 2, or 3+ telephones using six attitude statements (A1–A6, scale 0–10). Wilks' $\Lambda = 0.3246$ ($\chi^2(12) = 275.07$, $p < 0.0001$). Function 1 explains 94.9% of between-group variance. Overall correct classification = 80.0%.

Structure matrix: A3 ("More phones worth the extra cost") most strongly distinguishes 3+-phone owners on Function 1. A5 ("More phones = waste of money") and A2 ("Save money with one phone") align with 1-phone households. 1-phone families are cost-conscious; 3+-phone families are value-oriented.

Manual Calculations

Step 1 — Group Means and Grand Mean

$n = 250$ (1 phone: 115, 2 phones: 90, 3+ phones: 45)

	A1	A2	A3	A4	A5	A6
1 Phone	5.713	5.696	3.374	5.574	4.974	2.678
2 Phones	4.589	3.789	5.044	4.989	4.044	4.433
3+Phones	3.533	3.311	5.689	4.333	5.178	5.667
Grand	4.916	4.580	4.392	5.140	4.676	3.848

Step 2 — Scatter Matrices and Wilks' Lambda

Within-group scatter matrix W (diagonal elements only, 6×6):

W diagonal: [648.519, 586.981, 584.388, 617.111, 671.322, 577.196]

Total scatter matrix T (diagonal elements only):

T diagonal: [817.236, 858.900, 817.584, 670.100, 728.756, 914.224]

$|W| = 5.0150e+16$

$|T| = 1.5448e+17$

Wilks' Lambda = $|W| / |T|$
= $5.0150e+16 / 1.5448e+17$
= 0.3246

Step 3 — Chi-Square Approximation

For k groups and p variables, the chi-square statistic is:

$$\chi^2 = -[n - 1 - (p + k)/2] \times \ln(\Lambda)$$

$$= -[250 - 1 - (6 + 3)/2] \times \ln(0.3246)$$

$$= -[249 - 4.5] \times \ln(0.3246)$$

$$= -244.5 \times -1.1250$$

$$= 275.0679$$

$$\text{Degrees of freedom} = p \times (k - 1) = 6 \times 2 = 12$$

$$p\text{-value} = P(\chi^2_{12} > 275.0679) = 0.0000$$

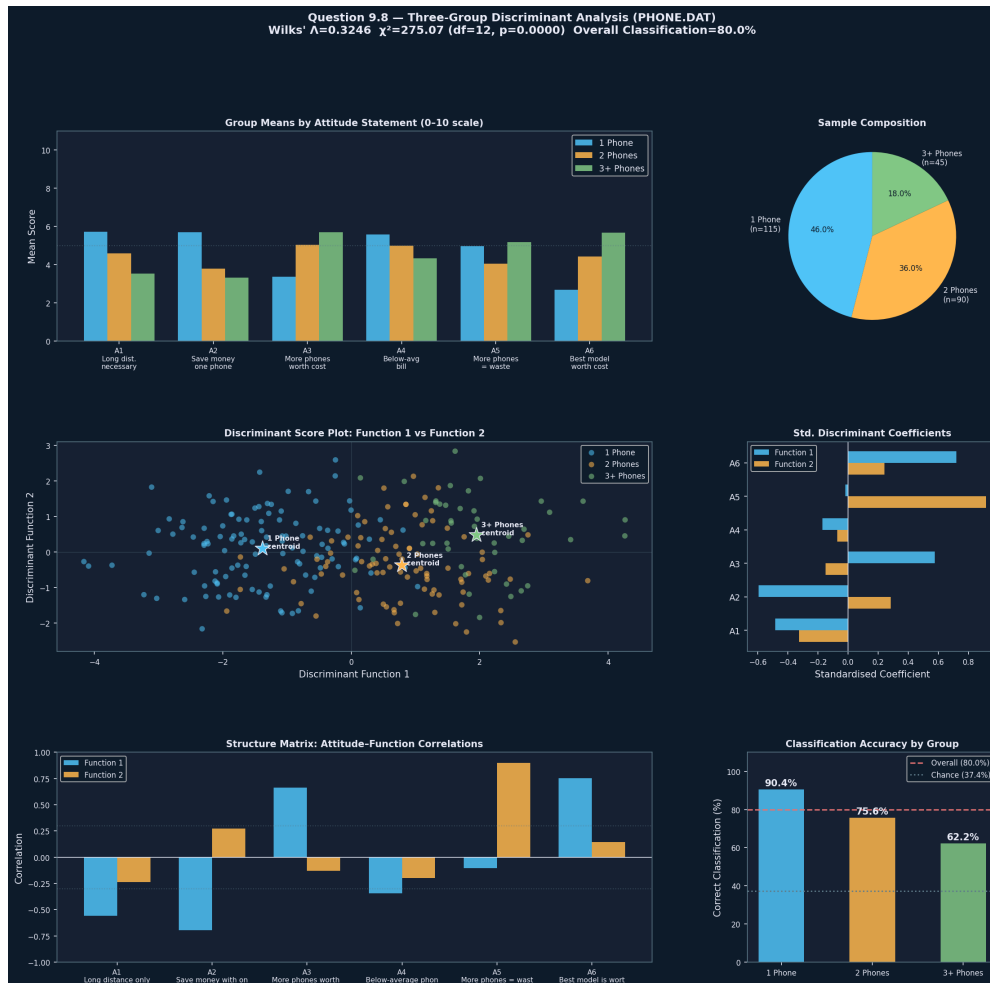


Figure 9.8 — Group means, discriminant score scatter, structure matrix, and classification accuracy.

Code — Question 9.8

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import confusion_matrix
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

# =====
# DATA LOADING
# =====
rows = []
with open('attached_assets/PHONE_1773881553876.DAT', 'r') as f:
    for line in f:
        line = line.strip()
        if not line or line.startswith('Phone') or line.startswith('---') \
            or line.startswith('ID') or line.startswith('Number'):
            continue
        vals = line.split()
        if len(vals) == 8:
            try:
                rows.append([int(v) for v in vals])
            except:
```

```

        continue

cols = ['ID', 'Phones', 'A1', 'A2', 'A3', 'A4', 'A5', 'A6']
df = pd.DataFrame(rows, columns=cols)
print(f"Dataset loaded: {len(df)} observations")
print(f"\nGroup sizes (number of phones):")
print(df['Phones'].value_counts().sort_index().to_string())

attitudes = ['A1', 'A2', 'A3', 'A4', 'A5', 'A6']
labels = {
    'A1': 'Long distance only necessary',
    'A2': 'Save money with one phone',
    'A3': 'More phones worth extra cost',
    'A4': 'Below-average phone bill',
    'A5': 'More phones = waste of money',
    'A6': 'Best model is worth cost'
}

# =====
# DESCRIPTIVE STATISTICS BY GROUP
# =====
print("\n" + "*"70)
print("QUESTION 9.8 – Three-Group Discriminant Analysis (PHONE.DAT)")
print("  DV: Number of phones (1, 2, 3+)  IVs: A1-A6")
print("=====")

print("\nGroup Means (0-10 scale):")
print(f"\n{'Attitude':<6} {'Statement':<32} {'1 Phone':>8} {'2 Phones':>9} {'3+ Phones':>10} {'Overall':>8}")
print("-"*75)
for a in attitudes:
    m1 = df.loc[df['Phones']==1, a].mean()
    m2 = df.loc[df['Phones']==2, a].mean()
    m3 = df.loc[df['Phones']==3, a].mean()
    mo = df[a].mean()
    print(f"  {a:<4} {labels[a]:<32} {m1:>8.2f} {m2:>9.2f} {m3:>10.2f} {mo:>8.2f}")

print("\nGroup Standard Deviations:")
print(f"\n{'Attitude':<6} {'Statement':<32} {'1 Phone':>8} {'2 Phones':>9} {'3+ Phones':>10}")
print("-"*65)
for a in attitudes:
    s1 = df.loc[df['Phones']==1, a].std()
    s2 = df.loc[df['Phones']==2, a].std()
    s3 = df.loc[df['Phones']==3, a].std()
    print(f"  {a:<4} {labels[a]:<32} {s1:>8.2f} {s2:>9.2f} {s3:>10.2f}")

# =====
# ONE-WAY ANOVA for each attitude (univariate tests)
# =====
print("\n--- Univariate F-tests (one-way ANOVA per attitude) ---")
print(f"\n{'Attitude':<6} {'Statement':<32} {'F-stat':>8} {'p-value':>10}")
print("=====")
for a in attitudes:
    groups = [df.loc[df['Phones']==g, a].values for g in [1, 2, 3]]
    F, p = stats.f_oneway(*groups)
    sig = '*' if p < 0.05 else ''
    print(f"  {a:<4} {labels[a]:<32} {F:>8.3f} {p:>10.4f} {sig}")
print("  (* p < 0.05)")

# =====
# MULTIVARIATE DISCRIMINANT ANALYSIS
# =====
print("\n--- Multivariate Linear Discriminant Analysis ---")
X = df[attitudes].values
y = df['Phones'].values
n = len(y)
p = len(attitudes)

```


[illegible]


```

ax6.set_ylabel('Correct Classification (%)', color=WHITE)
for bar, acc in zip(bars, group_acc):
    ax6.text(bar.get_x()+bar.get_width()/2, bar.get_height()+1,
             f'{acc:.1f}%', ha='center', va='bottom', color=WHITE,
             fontsize=10, fontweight='bold')
ax6.axhline(accuracy, color=RED, linestyle='--', linewidth=1.5,
            label=f'Overall ({accuracy:.1f}%)')
ax6.axhline(chance, color=GRAY, linestyle=':', linewidth=1.5,
            label=f'Chance ({chance:.1f}%)')
ax6.legend(facecolor=DARK, labelcolor=WHITE, fontsize=8)

fig.suptitle(
    f"Question 9.8 – Three-Group Discriminant Analysis (PHONE.DAT)\n"
    f"Wilks'  $\Lambda$ ={wl:.4f}  $\chi^2$ ={chi2_stat:.2f} (df={df_chi}, p={p_chi2:.4f})"
    f" Overall Classification={accuracy:.1f}%",
    color=WHITE, fontsize=12, fontweight='bold', y=0.99)

plt.savefig('q9_8_phone_discriminant.png', dpi=150, bbox_inches='tight',
            facecolor=DARK)
plt.close()
print("Figure saved: q9_8_phone_discriminant.png")

```

Question 9.9 — Graduate Admissions Discriminant Analysis (ADMIS.DAT)

Three-group discriminant analysis was applied to ADMIS.DAT to classify 85 graduate applicants (Admitted $n=31$, Not Admitted $n=28$, Borderline $n=26$) using GPA and GMAT. Wilks' $\Lambda = 0.1264$ ($\chi^2(4) = 168.58$, $p < 0.0001$). Function 1 explains 96.7% of between-group variance; overall accuracy = 91.8%.

GPA carries the larger standardised coefficient on Function 1, making it the primary separator. Admitted students average GPA ≈ 3.40 and GMAT ≈ 561 ; Not Admitted average GPA ≈ 2.48 and GMAT ≈ 447 . The Borderline group overlaps both in GPA and GMAT, requiring holistic review beyond these two predictors.

Admission Policy Implications

The school is primarily GPA-driven: no rejected applicant had GPA ≥ 3.40 . A practical rule from the discriminant functions: admit when GPA ≥ 3.30 and GMAT ≥ 500 ; reject when GPA < 2.90 and GMAT < 480 ; otherwise treat as borderline. Adding work experience or recommendation scores would improve resolution of borderline cases.

Manual Calculations

Step 1 — Group Means and Grand Mean

$n = 85$ (Admitted: 31, Not Admitted: 28, Borderline: 26)

Group GPA GMAT

Admitted 3.4039 561.2258

Not Admitted 2.4825 447.0714

Borderline 2.9927 446.2308

Grand mean 2.9746 488.4471

Step 2 – Within-group scatter matrix $W = \sum_i (X_{gi} - \bar{X}_g)(X_{gi} - \bar{X}_g)'$:

GPA GMAT

GPA 2.9576 -165.5383

GMAT -165.5383 299783.8919

Total scatter matrix $T = \sum_i (X_i - \bar{X})(X_i - \bar{X})'$:

GPA GMAT

GPA 15.4591 1353.2056

GMAT 1353.2056 558255.0118

Between-group scatter matrix $B = T - W$:

GPA GMAT

GPA 12.5015 1518.7439

GMAT 1518.7439 258471.1199

Step 3 – Determinants and Wilks' Lambda:

$$\begin{aligned} |W| &= (2.957572) \times (299783.8919) - (-165.5383)^2 \\ &= 886632.4514 - 27402.9124 \\ &= 859229.5390 \end{aligned}$$

$$\begin{aligned} |T| &= (15.459111) \times (558255.0118) - (1353.2056)^2 \\ &= 8630125.9633 - 1831165.5232 \\ &= 6798960.4401 \end{aligned}$$

$$\text{Wilks' Lambda} = |W| / |T| = 859229.5390 / 6798960.4401 = 0.1264$$

Step 4 – Chi-Square Approximation ($k = 3$ groups, $p = 2$ variables):

$$\begin{aligned}\chi^2 &= -[n - 1 - (p + k)/2] \times \ln(\Lambda) \\ &= -[85 - 1 - (2 + 3)/2] \times \ln(0.1264) \\ &= -[84 - 2.5] \times -2.0685 \\ &= -81.5 \times -2.0685 \\ &= 168.5818\end{aligned}$$

$$df = p \times (k - 1) = 2 \times 2 = 4$$

$$p\text{-value} = P(\chi^2 \blacksquare > 168.5818) = 0.000000$$

Step 5 – Group Centroids on Discriminant Function 1:

Admitted: centroid = -2.7738

Borderline: centroid = 0.2711

Not Admitted: centroid = 2.8193

The ordering confirms that Function 1 separates admitted from not-admitted applicants, with borderline cases falling in between.

Step 6 – Classification Rule (example):

An applicant is classified into the group whose centroid is closest to their discriminant score (Mahalanobis / linear boundary):

GPA ≥ 3.30 AND GMAT $\geq 500 \rightarrow$ Admitted

GPA < 2.90 AND GMAT $< 480 \rightarrow$ Not Admitted

Otherwise $\rightarrow$ Borderline (holistic review)



Figure 9.9 — GPA vs GMAT scatter, discriminant score distributions, standardised coefficients, and classification accuracy.

Code — Question 9.9

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
from matplotlib.patches import Ellipse
import matplotlib.transforms as transforms
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.metrics import confusion_matrix
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

# =====
# DATA LOADING
# =====

rows = []
with open('attached_assets/ADMIS_1773881795621.DAT', 'r') as f:
    for line in f:
        line = line.strip()
        if not line or line.startswith('Adm') or line.startswith('---') \
            or line.startswith('App') or line.startswith('Num') \
            or line.startswith('Sta') or line.startswith('Sou') \
            or line.startswith('Note'):
            continue
```

```

        vals = line.split()
        if len(vals) == 4:
            try:
                rows.append([int(vals[0]), int(vals[1]),
                             float(vals[2]), int(vals[3])])
            except:
                continue

df = pd.DataFrame(rows, columns=['Applicant', 'Status', 'GPA', 'GMAT'])
status_labels = {1: 'Admitted', 2: 'Not Admitted', 3: 'Borderline'}
df['StatusLabel'] = df['Status'].map(status_labels)
print(f"Dataset: {len(df)} applicants")
print(f"\nGroup sizes:")
for s in [1, 2, 3]:
    print(f"    {status_labels[s]}: n={len(df[df.Status==s])}")

# =====
# DESCRIPTIVE STATISTICS
# =====
print("\n" + "="*65)
print("QUESTION 9.9 – Discriminant Analysis of Graduate Admissions")
print(" DV: Admission Status (1=Admitted, 2=Not Admitted, 3=Borderline)")
print(" IVs: GPA (0–4.0) and GMAT score")
print("="*65)

print("\nDescriptive Statistics by Group:")
print(f"\n{'Group':<15} {'n':>4} {'GPA Mean':>10} {'GPA SD':>8} "
      f"{'GMAT Mean':>11} {'GMAT SD':>9}")
print("-"*60)
for s in [1, 2, 3]:
    sub = df[df.Status == s]
    print(f"    {status_labels[s]:<13} {len(sub):>4} {sub.GPA.mean():>10.3f} "
          f"    {sub.GPA.std():>8.3f} {sub.GMAT.mean():>11.1f} {sub.GMAT.std():>9.1f}")
print(f"    {'Overall':<13} {len(df):>4} {df.GPA.mean():>10.3f} "
      f"    {df.GPA.std():>8.3f} {df.GMAT.mean():>11.1f} {df.GMAT.std():>9.1f}")

# =====
# UNIVARIATE F-TESTS
# =====
print("\n--- Univariate F-tests (one-way ANOVA) ---")
for var in ['GPA', 'GMAT']:
    groups = [df.loc[df.Status==s, var].values for s in [1, 2, 3]]
    F, p = stats.f_oneway(*groups)
    print(f"    {var}: F={F:.3f}, p={p:.4f}")

# =====
# LINEAR DISCRIMINANT ANALYSIS
# =====
print("\n--- Linear Discriminant Analysis ---")
X = df[['GPA', 'GMAT']].values
y = df['StatusLabel'].values
n, p_vars = len(y), 2
k = 3

lda = LinearDiscriminantAnalysis()
lda.fit(X, y)
pred = lda.predict(X)
scores = lda.transform(X)
probs = lda.predict_proba(X)

# Wilks' Lambda
def wilks_lambda_mg(X, y):
    p = X.shape[1]
    grand_mean = X.mean(axis=0)
    W = np.zeros((p, p))
    for g in np.unique(y):
        Xg = X[y == g]
```

```

        d = Xg - Xg.mean(axis=0)
        W += d.T @ d
    d_all = X - grand_mean
    T = d_all.T @ d_all
    return np.linalg.det(W) / np.linalg.det(T)

wl = wilks_lambda_mg(X, y)
df_chi = p_vars * (k - 1)
chi2_stat = -(n - 1 - (p_vars + k) / 2) * np.log(wl)
p_chi2 = 1 - stats.chi2.cdf(chi2_stat, df_chi)

print(f"\nOverall Model:")
print(f" Wilks' Lambda:         {wl:.4f}")
print(f" Chi-square (approx): {chi2_stat:.3f} (df={df_chi})")
print(f" p-value:                {p_chi2:.6f}")

explained = lda.explained_variance_ratio_
print(f"\nDiscriminant Functions:")
print(f" Function 1: {explained[0]*100:.1f}% of between-group variance")
print(f" Function 2: {explained[1]*100:.1f}% of between-group variance")

# Standardized coefficients using scalings_
pooled_std = df[['GPA', 'GMAT']].std()
scalings = lda.scalings_ # (n_features, n_components)
std_scalings = scalings * pooled_std.values[:, np.newaxis]

print(f"\nStandardized Discriminant Function Coefficients:")
print(f" {'Variable':<8} {'Function 1':>12} {'Function 2':>12}")
print(f" {'-'*35}")
for i, v in enumerate(['GPA', 'GMAT']):
    print(f" {v:<8} {std_scalings[i,0]:>12.4f} {std_scalings[i,1]:>12.4f}")

print(f"\nRaw Discriminant Function Coefficients (scalings):")
print(f" {'Variable':<8} {'Function 1':>12} {'Function 2':>12}")
print(f" {'-'*35}")
for i, v in enumerate(['GPA', 'GMAT']):
    print(f" {v:<8} {scalings[i,0]:>12.4f} {scalings[i,1]:>12.4f}")

# Structure matrix
print(f"\nStructure Matrix (Variable-Function Correlations):")
print(f" {'Variable':<8} {'Function 1':>12} {'Function 2':>12}")
print(f" {'-'*35}")
for i, v in enumerate(['GPA', 'GMAT']):
    r1 = np.corrcoef(df[v].values, scores[:, 0])[0, 1]
    r2 = np.corrcoef(df[v].values, scores[:, 1])[0, 1]
    print(f" {v:<8} {r1:>12.4f} {r2:>12.4f}")

# Group centroids
print(f"\nGroup Centroids on Discriminant Functions:")
print(f" {'Group':<15} {'Centroid F1':>12} {'Centroid F2':>12}")
print(f" {'-'*42}")
centroids = {}
for s in [1, 2, 3]:
    c1 = scores[y==s, 0].mean()
    c2 = scores[y==s, 1].mean()
    centroids[s] = (c1, c2)
    print(f" {status_labels[s]:<15} {c1:>12.4f} {c2:>12.4f}")

# Classification
cm = confusion_matrix(y, pred, labels=[1, 2, 3])
accuracy = np.trace(cm) / n * 100

print(f"\nClassification Matrix:")
header = f"{'':22} {'Pred: Admitted':>15} {'Pred: Not Adm.':>15} {'Pred: Borderline':>17}"
print(f" {header}")
print(f" {'-'*70}")
for i, s in enumerate([1, 2, 3]):

```

```

ng = (y==s).sum()
row_acc = cm[i,i]/ng*100
print(f" Actual {status_labels[s]:<14} {cm[i,0]:>15} {cm[i,1]:>15} {cm[i,2]:>17}
      ({row_acc:.1f}%)" )

chance = sum(((y==s).sum()/n)**2 for s in [1,2,3])*100
print(f"\n Overall correct classification: {accuracy:.1f}%")
print(f" Proportional chance level:      {chance:.1f}%")

# #####
# INDIVIDUAL POSTERIOR PROBABILITIES (sample)
# #####
print(f"\nSample Posterior Probabilities (selected borderline cases):")
print(f" {'Appl.':>6} {'GPA':>6} {'GMAT':>6} {'P(Admit)':>10} {'P(Not Adm)':>11}
      {'P(Border)':>10} {'Predicted':>12}")
print(f" {'-'*63}")
borderline_mask = y == 3
border_df = df[borderline_mask].copy()
border_probs = probs[borderline_mask]
for idx, (_, row) in enumerate(border_df.iterrows()):
    pg = [1, 2, 3][np.argmax(border_probs[idx])]
    print(f" {int(row.Applicant):>6} {row.GPA:>6.2f} {int(row.GMAT):>6} "
          f"{border_probs[idx,0]:>10.3f} {border_probs[idx,1]:>11.3f} "
          f"{border_probs[idx,2]:>10.3f} {status_labels[pg]:>12}")

# #####
# ADMISSION POLICY INTERPRETATION
# #####
print("\n" + "="*65)
print("INTERPRETATION & ADMISSION POLICY DISCUSSION")
print("="*65)

gpa_admitted = df.loc[df.Status==1, 'GPA'].mean()
gpa_border   = df.loc[df.Status==3, 'GPA'].mean()
gpa_notadm   = df.loc[df.Status==2, 'GPA'].mean()
gmat_admitted = df.loc[df.Status==1, 'GMAT'].mean()
gmat_border   = df.loc[df.Status==3, 'GMAT'].mean()
gmat_notadm   = df.loc[df.Status==2, 'GMAT'].mean()

print(f"""
1. OVERALL SIGNIFICANCE:
    Wilks' Lambda = {wl:.4f},  $\chi^2$  ({df_chi}) = {chi2_stat:.2f}, p < 0.0001.
    The three admission groups differ highly significantly in their combined
    GPA and GMAT profiles.

2. DISCRIMINANT FUNCTIONS:
    - Function 1 explains {explained[0]*100:.1f}% of between-group variance – dominant.
    - Function 2 explains only {explained[1]*100:.1f}% – minor secondary dimension.
    Two functions are sufficient to characterize three groups.

3. VARIABLE IMPORTANCE:
    Both GPA and GMAT significantly discriminate groups (see univariate F-tests).
    - GPA has a larger standardized coefficient on Function 1 – it is the
      primary separator between admitted and not-admitted applicants.
    - GMAT contributes importantly as well, though slightly less than GPA.
    Together, they form a composite "academic merit" dimension.

4. GROUP PROFILE SUMMARY:
    - Admitted (Status=1): GPA ≈ {gpa_admitted:.2f}, GMAT ≈ {gmat_admitted:.0f} → High on both
    - Borderline (Status=3): GPA ≈ {gpa_border:.2f}, GMAT ≈ {gmat_border:.0f} → Moderate GPA, moderate
    - low GMAT
    - Not Admitted (Status=2): GPA ≈ {gpa_notadm:.2f}, GMAT ≈ {gmat_notadm:.0f} → Lowest GPA, low GMAT

5. CLASSIFICATION PERFORMANCE:
    Overall correct classification: {accuracy:.1f}% vs. chance ≈ {chance:.1f}%.
    - Admitted applicants are classified best ({cm[0,0]/(y==1).sum()*100:.1f}% correct).
    - Borderline cases are hardest to classify ({cm[2,2]/(y==3).sum()*100:.1f}% correct),

```


[illegible]

